

Міністерство освіти і науки України

Відокремлений структурний підрозділ «Тернопільський фаховий коледж
Тернопільського національного технічного університету імені Івана Пулюя»

(повне найменування вищого навчального закладу)

відділення телекомунікацій та електронних систем

(назва відділення)

циклова комісія комп'ютерних наук

(повна назва циклової комісії)

ПОЯСНЮВАЛЬНА ЗАПИСКА до курсової роботи

на тему: Розробка веб-додатку для книжкового онлайн-
магазину

Виконав: студент (ка) 3 курсу, групи КН-321

Спеціальності: 122 «Комп'ютерні науки»

Павло ІВАШКІВ

(ім'я та прізвище)

Керівник Володимир ЛІСОВИЙ

(ім'я та прізвище)

Оцінка: _____

Члени комісії: _____

(підпис)

(ім'я та прізвище)

(підпис)

(ім'я та прізвище)

ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ «ТЕРНОПІЛЬСЬКИЙ
ФАХОВИЙ КОЛЕДЖ ТЕРНОПІЛЬСЬКОГО НАЦІОНАЛЬНОГО
ТЕХНІЧНОГО УНІВЕРСИТЕТУ ІМЕНІ ІВАНА ПУЛЮЯ»

Відділення телекомунікацій та електронних систем

Циклова комісія комп'ютерних наук

Освітньо-професійний ступінь фаховий молодший бакалавр

Спеціальність 122 «Комп'ютерні науки»

(шифр і назва)

Галузь знань 12 «Інформаційні технології»

(шифр і назва)

ЗАТВЕРДЖУЮ

Голова циклової комісії
комп'ютерних наук

_____ Галина МАРЦІЯШ
“ 30 ” березня 2026 року

**ЗАВДАННЯ
НА КУРСОВУ РОБОТУ СТУДЕНТУ**

Павлу ІВАШКІВУ
(ім'я та прізвище)

1. Тема роботи:

_____ Розробка веб-додатку для книжкового онлайн-магазину _____

2. Керівник роботи:

Володимир ЛІСОВИЙ
(ім'я та прізвище)

3. Строк подання студентом роботи: 12 червня 2026р _____

4. Вихідні дані до роботи Інформація по темі курсової роботи, мови HTML5, CSS3, JavaScript, NodeJS, Express JS, React JS, стандарти мов програмування

5. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити)

ВСТУП

1 ЗАГАЛЬНИЙ РОЗДІЛ

1.1 Аналітичний огляд існуючих рішень

1.2 Технічне завдання

2 РОЗРОБКА ТЕХНІЧНОГО ТА РОБОЧОГО ПРОЕКТУ

2.1 Розробка структури сайту і web-сторінок

2.2 Створення та верстка сторінок сайту

2.3 Розробка структури бази даних сайту

2.4 Написання JavaScript кодів

2.5 Тестування web-сайту

3 СПЕЦІАЛЬНИЙ РОЗДІЛ

3.1 Інструкція з розміщення сайту в Інтернеті

3.2 Інструкція з обслуговування та наповнення сайту

3.3 Інструкція з популяризації та підтримки сайту

ВИСНОВКИ

ПЕРЕЛІК ПОСИЛАНЬ

ДОДАТКИ

Додаткові вказівки: виконання проекту з написанням сайту та розміщенням його в мережі інтернет і вихідних кодів в github

Дата видачі завдання « 30 » 03 2026 р.

Студент

(підпис)

Павло ІВАШКІВ

(ім'я та прізвище)

Керівник роботи

(підпис)

Володимир ЛІСОВИЙ

(ім'я та прізвище)

ЗМІСТ

ВСТУП	5
1 ЗАГАЛЬНИЙ РОЗДІЛ	7
1.1 Аналітичний огляд існуючих рішень	7
1.2 Технічне завдання.....	8
1.2.1 Найменування та галузь застосування	8
1.2.2 Підстави для розробки	8
1.2.3 Призначення та мета розробки	9
1.2.4 Вимоги до функціональних характеристик.....	9
1.2.5 Вимоги до програмного та технічного забезпечення	10
2 РОЗРОБКА ТЕХНІЧНОГО ТА РОБОЧОГО ПРОЕКТУ	11
2.1 Розробка структури сайту і веб-сторінок	11
2.2 Створення та верстка сторінок сайту	13
2.3 Розробка структури бази даних.....	15
2.4 Програмна реалізація.....	16
2.5 Тестування веб-сайту	21
3 СПЕЦІАЛЬНИЙ РОЗДІЛ	23
3.1 Інструкція з розміщення сайту в мережі Інтернет	23
3.2 Інструкція з обслуговування та наповнення сайту	23
3.3 Інструкція з популяризації та підтримки сайту.....	24
ВИСНОВКИ.....	25
ПЕРЕЛІК ПОСИЛАНЬ	26
ДОДАТОК А.....	26
ДОДАТОК Б	31
ДОДАТОК В.....	39

					2026.КР.122.321.10.00.00 ПЗ						
Зм.	Арк.	№ докум.	Підпис	Дата							
Розробив		Івашків П.В.			«Розробка веб-додатку для книжкового онлайн-магазину»			Літ.	Арк.	Аркушів	
Перевірів		Лісовий В.М.								4	44
Рецензент								ВСП ТФК ТНТУ КН-321 м. Тернопіль			
Н. контр.											
Затвердив											

ВСТУП

Останніми роками купівля товарів через мережу Інтернет стала звичною частиною повсякденного життя. Книжкова торгівля не лишилася осторонь: читачі дедалі частіше обирають і замовляють літературу онлайн, адже це економить час, не залежить від місця проживання та дає змогу швидко відшукати потрібне видання серед тисяч позицій. Тому розробка веб-додатків для книжкових онлайн-магазинів залишається затребуваним напрямом сучасної веб-розробки.

На відміну від найпростіших навчальних сторінок, повноцінний інтернет-магазин складається з двох частин: клієнтської, з якою працює відвідувач, та серверної, що зберігає товари в базі даних і обробляє запити. Каталог книг повинен зберігатися централізовано, а всі дії з ним – додавання, редагування, видалення, пошук та сортування – мають виконуватися через звернення до сервера. Реалізація такого клієнт-серверного веб-додатку і є предметом цієї курсової роботи.

Метою курсової роботи є розробка веб-додатку книжкового онлайн-магазину «Літера» за технологією стеку MERN (MongoDB, Express, React, Node.js), який дає змогу переглядати каталог книг, шукати, фільтрувати та сортувати товари, гортати сторінки каталогу, а також керувати ним – додавати, змінювати й видаляти книги – зі збереженням усіх даних у базі даних на сервері.

Об'єктом дослідження є процес функціонування книжкового інтернет-магазину. Предметом дослідження є технології стеку MERN та принципи побудови клієнт-серверних веб-додатків із використанням REST API й бази даних.

Щоб досягти поставленої мети, у роботі вирішено такі завдання:

- оглянути та порівняти наявні книжкові інтернет-магазини, виокремивши типовий набір їхніх можливостей;
- скласти технічне завдання на розробку веб-додатку;

					2026.KP.122.321.10.00.00 ПЗ	Арк.
						5
Зм.	Арк.	№ докум.	Підпис	Дата		

- обрати технології для реалізації клієнтської та серверної частин;
- спроектувати структуру бази даних та REST API для роботи з каталогом;
- розробити інтерфейс користувача у вигляді односторінкового React-додатку;
- запрограмувати перегляд, пошук, фільтрування, сортування, пагінацію, додавання, редагування й видалення книг із перевіркою введених даних;
- провести випробування додатку та підготувати його до розміщення в мережі Інтернет.

Практична цінність роботи полягає у створенні завершеного клієнт-серверного веб-додатку, що демонструє ключові можливості каталогу інтернет-магазину з реальною базою даних і може слугувати підґрунтям для подальшого розвитку – зокрема для додавання кошика, особистих кабінетів та оформлення замовлень.

					2026.KP.122.321.10.00.00 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		6

1 ЗАГАЛЬНИЙ РОЗДІЛ

1.1 Аналітичний огляд існуючих рішень

Перш ніж приступати до проєктування, доцільно ознайомитися з тим, як побудовані наявні книжкові інтернет-магазини. Це дає змогу визначити перелік можливостей, які користувачі вважають звичними, та врахувати їх під час розробки власного додатку. Для огляду обрано три відомі українські ресурси.

Yaкабоо [1] – найбільший в Україні онлайн-магазин книг. Пропонує об'ємний каталог, докладні картки товарів, розгалужену систему фільтрів за категоріями, авторами й цінами, кошик та оформлення замовлення. Водночас сторінки перевантажені елементами та рекламними блоками, а для частини дій потрібна реєстрація.

Книгарня «Є» [2] – мережа книжкових магазинів із власним онлайн-майданчиком. Книги згруповано за розділами, передбачено пошук та опис кожного видання з обкладинкою. Інтерфейс простіший за попередній, проте поповнювати й змінювати каталог можуть лише адміністратори через закриту панель.

Розділ книг Rozetka – частина великого універсального маркетплейсу. Має потужні пошук і сортування, однак книги тут є лише однією з безлічі категорій товарів, тому оформлення не зосереджене саме на книжковій тематиці.

Спільним для всіх оглянутих майданчиків є подання товарів у вигляді карток із зображенням, назвою, ціною та категорією, наявність пошуку, сортування й посторінкового виведення товарів, а також зберігання каталогу на сервері. Узагальнене порівняння наведено в таблиці 1.1.

					2026.KP.122.321.10.00.00 ПЗ	Арк.
						7
Зм.	Арк.	№ докум.	Підпис	Дата		

Можливість	Yakaboo	Книгарня «Є»	Rozetka	«Літера»
Каталог-картки	+	+	+	+
Пошук за назвою	+	+	+	+
Сортування та фільтри	+	–	+	+
Пагінація	+	+	+	+
Керування каталогом користувачем	–	–	–	+
Робота без реєстрації	–	+	–	+

Таблиця 1.1 – Порівняння книжкових інтернет-магазинів

Огляд засвідчив, що наявні магазини є складними комерційними системами, у яких керування товарами приховане від звичайного відвідувача. Тому в межах курсової роботи доцільно створити простіший навчальний веб-додаток зі схожою архітектурою «клієнт – сервер – база даних», де безпосередньо на сторінці можна не лише переглядати книги, а й вільно змінювати каталог.

1.2 Технічне завдання

1.2.1 Найменування та галузь застосування

Найменування розробки – веб-додаток книжкового онлайн-магазину «Літера».

Галузь застосування – електронна торгівля книгами, а також навчальний процес із веб-технологій. Клієнтська частина додатку відкривається у будь-якому сучасному браузері й звертається до серверної частини через мережу.

1.2.2 Підстави для розробки

Підставою для розробки є завдання на курсову роботу з дисципліни «Web технології та Web дизайн» для спеціальності 122 «Комп’ютерні науки». За умовою потрібно розробити веб-додаток книжкового онлайн-магазину з такими категоріями товарів: художня література, наукова література, дитячі книги, комікси та навчальна література. Кожна книга описується назвою, описом, ціною, знижкою у відсотках, кількістю на складі, зображенням обкладинки й категорією.

1.2.3 Призначення та мета розробки

Призначення веб-додатку – надати зручний засіб для перегляду каталогу книг та керування ним. Користувач переглядає товари у вигляді карток, знаходить потрібні видання, фільтрує та сортує їх, гортає сторінки каталогу, а також додає нові книги, змінює їхні характеристики й вилучає зайві позиції.

Мета розробки – побудувати працездатний клієнт-серверний веб-додаток, що реалізує основні функції каталогу інтернет-магазину засобами стеку MERN зі збереженням даних у базі даних MongoDB.

1.2.4 Вимоги до функціональних характеристик

Веб-додаток має забезпечувати виконання таких функцій:

1. виведення переліку книг у вигляді карток із обкладинкою, назвою, описом, ціною, знижкою та кількістю на складі;
2. додавання нової книги через модальне вікно з формою;
3. редагування характеристик наявної книги в тому самому вікні;
4. видалення книги з обов'язковим підтвердженням дії;
5. пошук книг за назвою, описом або категорією;
6. фільтрування книг за обраною категорією;
7. сортування книг за ціною та за назвою;
8. посторінкове виведення каталогу (пагінацію);
9. перевірку (валідацію) введених даних як на клієнті, так і на сервері;
10. збереження всього каталогу в базі даних MongoDB.

Узагальнено можливості користувача під час роботи з веб-додатком унаочнює діаграма варіантів використання, наведена на рисунку 1.1.

					2026.KP.122.321.10.00.00 ПЗ	Арк.
						9
Зм.	Арк.	№ докум.	Підпис	Дата		

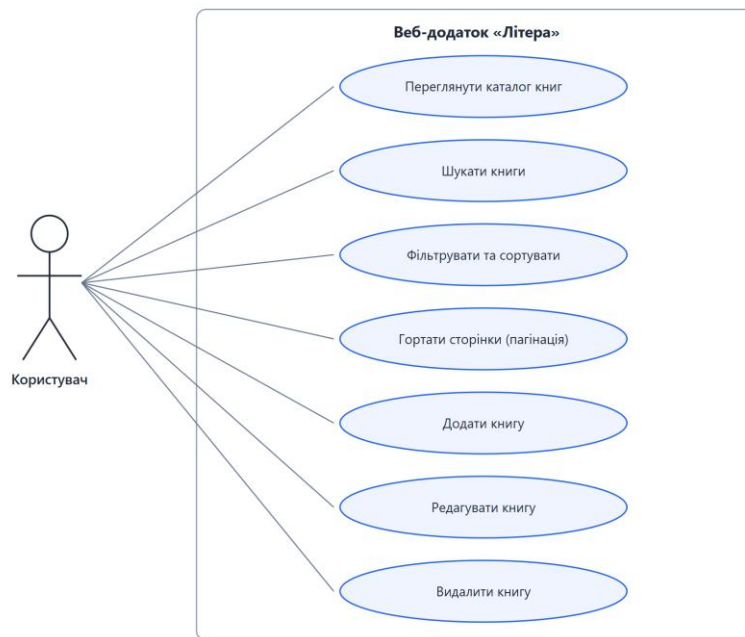


Рисунок 1.1 – Діаграма варіантів використання веб-додатку «Літера»

1.2.5 Вимоги до програмного та технічного забезпечення

Для роботи з клієнтською частиною додатку достатньо персонального комп'ютера, ноутбука, планшета чи смартфона із встановленим сучасним веб-браузером (Google Chrome, Microsoft Edge, Mozilla Firefox чи Opera) та підключенням до мережі Інтернет. Серверна частина потребує середовища виконання Node.js, а для зберігання даних – системи керування базами даних MongoDB. Для розробки використано персональний комп'ютер під керуванням ОС Windows та редактор коду Visual Studio Code.

2 РОЗРОБКА ТЕХНІЧНОГО ТА РОБОЧОГО ПРОЕКТУ

2.1 Розробка структури сайту і веб-сторінок

Для реалізації веб-додатку обрано стек технологій MERN, який дає змогу писати і клієнтську, і серверну частини однією мовою – JavaScript [3]. Цей стек складається з чотирьох основних компонентів та додаткових інструментів, перелік яких наведено на рисунку 2.1.

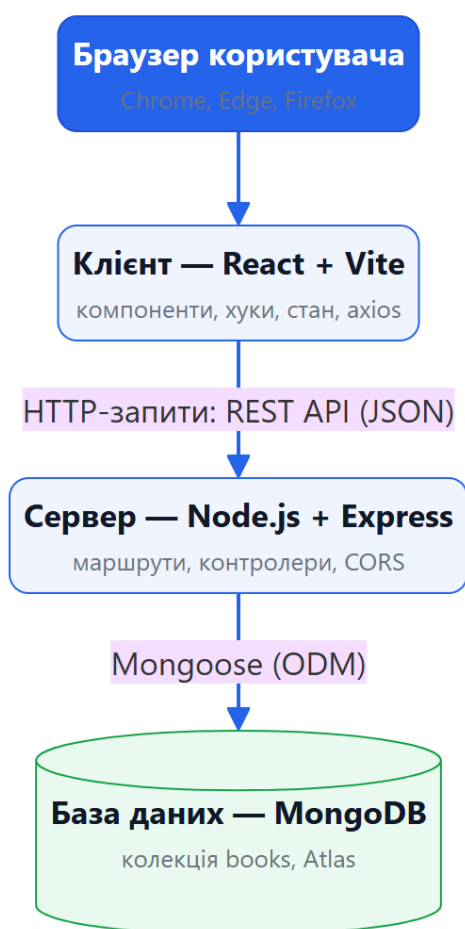


Рисунок 2.1 – Технології, застосовані у веб-додатку «Litera»

React [4] – бібліотека для побудови інтерфейсу користувача з окремих компонентів; разом зі складальником Vite [5] вона утворює клієнтську частину. Node.js [6] з фреймворком Express [7] відповідає за серверну частину та REST API. MongoDB [8] – документоорієнтована база даних, у якій

зберігається каталог, а бібліотека Mongoose [9] описує схему даних і спрощує роботу з нею. Обмін даними між клієнтом і сервером відбувається у форматі JSON за допомогою HTTP-бібліотеки axios [10].

Додаток має класичну трирівневу архітектуру: браузер користувача звертається до клієнтської React-частини, та через REST API надсилає запити на сервер Express, який, своєю чергою, читає й записує дані в базу MongoDB. Проєкт поділено на дві папки – backend і frontend, кожна з яких відповідає за свою частину. Склад файлів проєкту наведено на рисунку 2.2.

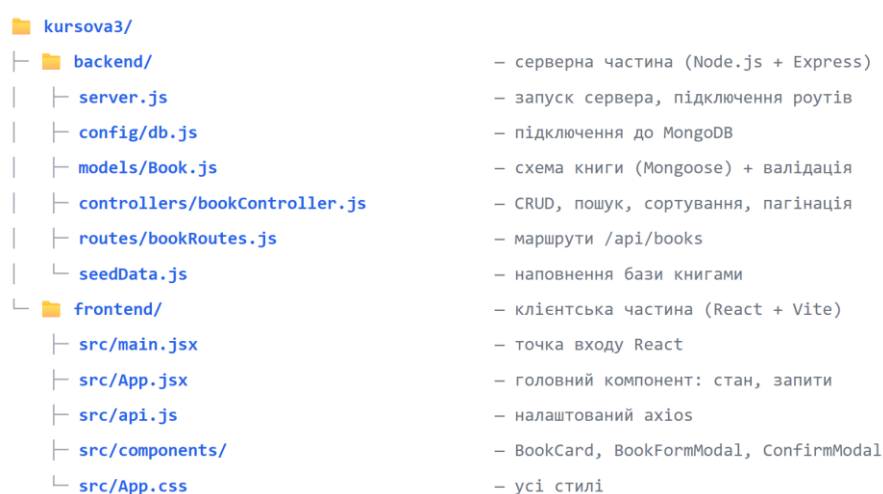


Рисунок 2.2 – Склад файлів проєкту

Папка backend містить серверну частину: файл server.js запускає сервер Express, config/db.js підключається до бази даних, models/Book.js описує схему книги, controllers/bookController.js містить логіку обробки запитів, а routes/bookRoutes.js задає маршрути API. Папка frontend містить клієнтську частину: головний компонент App.jsx, окремі компоненти карток і модальних вікон у папці components, налаштований axios у файлі api.js та стилі у файлі App.css.

Загальну структуру додатку з поділом на клієнтську й серверну частини подано у вигляді структурної схеми на рисунку 2.3.

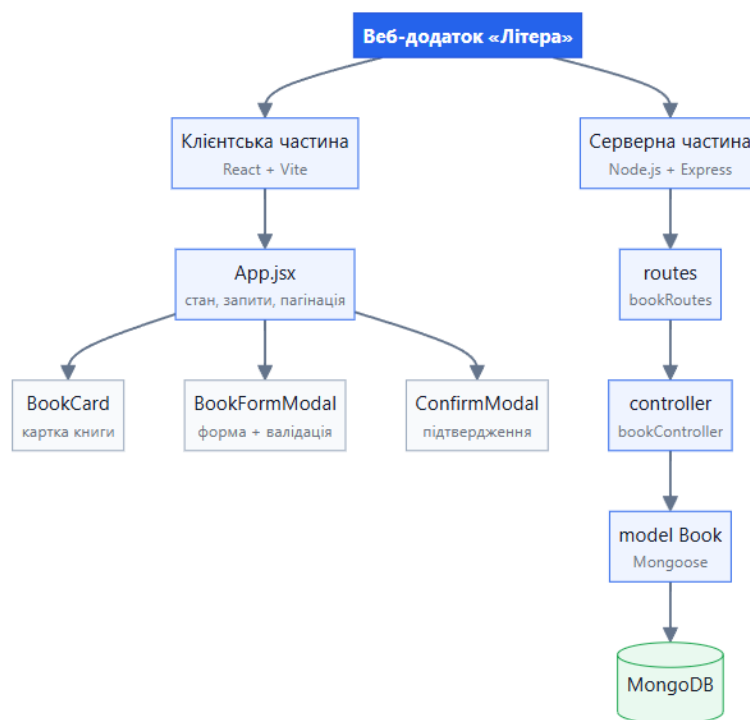


Рисунок 2.3 – Структурна схема веб-додатку «Літера»

2.2 Створення та верстка сторінок сайту

Клієнтську частину побудовано як односторінковий додаток (SPA) на бібліотеці React: уся взаємодія відбувається на одній сторінці, а потрібні фрагменти інтерфейсу перемальовуються без її повного перезавантаження. Інтерфейс поділено на самостійні компоненти: головний компонент App відповідає за загальну будову сторінки та стан додатку, компонент BookCard виводить картку окремої книги, BookFormModal реалізує форму додавання й редагування, а ConfirmModal – вікно підтвердження видалення.

У верхній частині сторінки розташовано шапку з назвою магазину та кнопкою додавання книги, нижче – панель із полем пошуку й двома випадними списками (для фільтрування за категорією та для впорядкування), а основну площу займає сітка карток книг. Внизу сторінки виводяться кнопки посторінкового переходу.

Оформлення сторінки реалізовано у файлі стилів App.css звичайним CSS. Картки книг побудовано за допомогою механізму CSS Grid, що автоматично розподіляє їх у рядки й стовпці залежно від ширини вікна. Адаптивність забезпечено медіазапитами: на широкому екрані картки шикуються в чотири стовпці, на планшеті – у два, а на смартфоні – в один. Кожна картка містить обкладинку, назву, категорію, опис, ціну з урахуванням знижки, відомості про наявність та кнопки редагування й видалення. Загальний вигляд готової головної сторінки показано на рисунку 2.4.

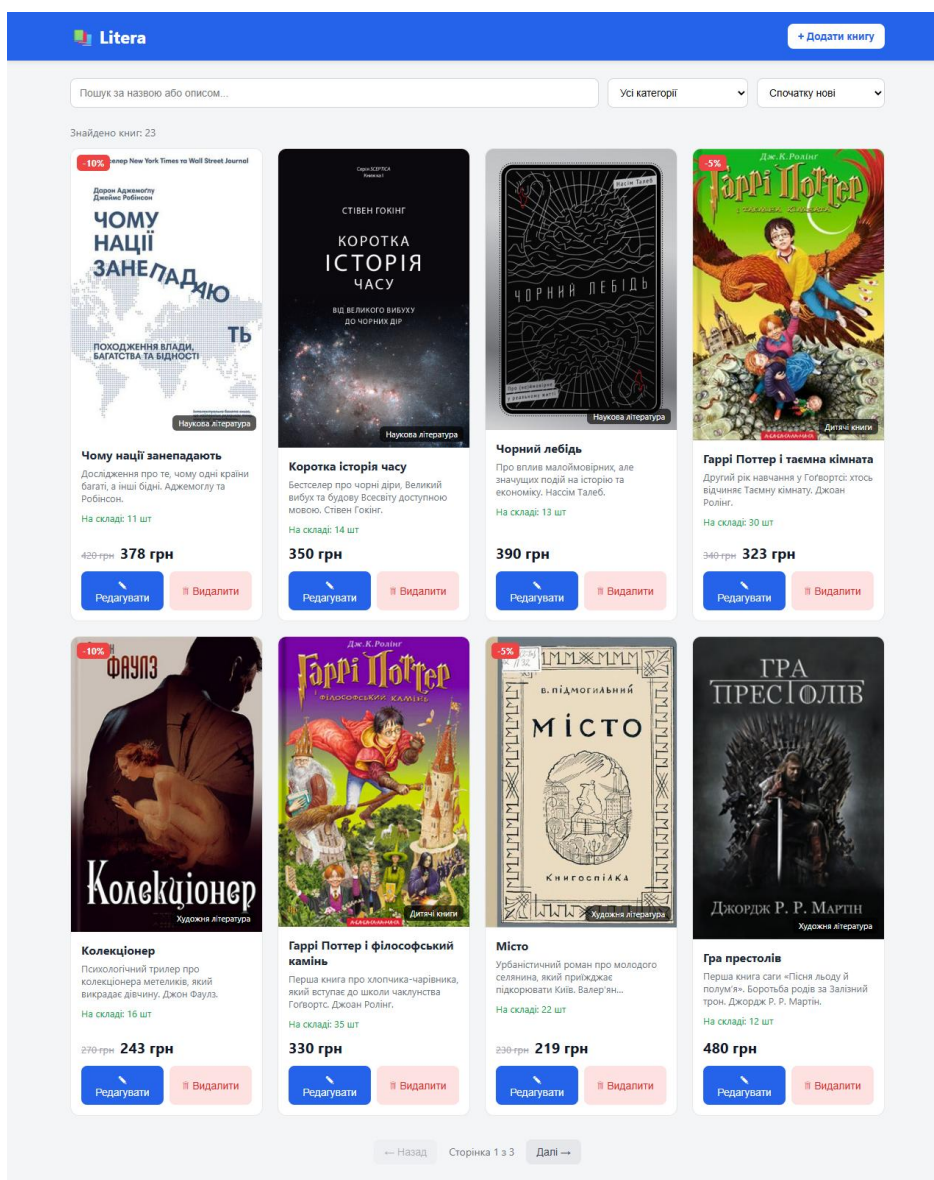


Рисунок 2.4 – Загальний вигляд головної сторінки веб-додатку

					2026.KP.122.321.10.00.00 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		14

2.3 Розробка структури бази даних

Дані про книги зберігаються в документоорієнтованій базі даних MongoDB у колекції books. Кожна книга є окремим документом у форматі, близькому до JSON. Структуру документа описано схемою Mongoose у файлі models/Book.js; ця схема також задає правила перевірки даних на стороні сервера. Поля документа книги наведено на рисунку 2.5.

Колекція books — схема Mongoose (MongoDB)

Поле	Тип	Призначення
<code>_id</code>	ObjectId	унікальний ідентифікатор (генерує MongoDB)
<code>title</code>	String	назва книги (обов'язкове)
<code>description</code>	String	опис книги (обов'язкове)
<code>price</code>	Number	ціна, грн (не менше 0)
<code>discount</code>	Number	знижка, % (від 0 до 100)
<code>stock</code>	Number	кількість на складі (не менше 0)
<code>image</code>	String	посилання на обкладинку
<code>category</code>	String	категорія (enum — 5 значень)
<code>createdAt</code>	Date	дата створення (timestamps)
<code>updatedAt</code>	Date	дата останнього оновлення (timestamps)

Рисунок 2.5 – Структура документа книги (схема Mongoose)

Поле `_id` є унікальним ідентифікатором документа й автоматично створюється базою даних; саме за ним відшукується потрібний запис під час редагування або видалення. Решта полів відповідають характеристикам товару із завдання: `title` – назва, `description` – опис, `price` – ціна, `discount` – знижка у відсотках, `stock` – кількість на складі, `image` – посилання на обкладинку та `category` – категорія. Для кожного поля у схемі задано тип і обмеження: назва, опис, ціна, кількість і категорія є обов'язковими, ціна та кількість не можуть бути від'ємними, знижка обмежена діапазоном від 0 до 100, а категорія може набувати лише одного з п'яти дозволених значень. Завдяки увімкненим часовим міткам (timestamps) для кожного документа автоматично

зберігаються дати створення та оновлення. Початкове наповнення бази виконує скрипт `seedData.js`, який записує до неї 23 книги п'яти категорій.

2.4 Програмна реалізація

Серверна частина реалізує REST API – набір маршрутів, до яких звертається клієнт. Усі маршрути починаються з адреси `/api/books` і обробляють книги: отримання списку, отримання однієї книги, створення, оновлення та видалення. Маршрути описано у файлі `bookRoutes.js`, а їхню логіку – у контролері `bookController.js`. Запити до бази даних виконуються асинхронно за допомогою конструкції `async/await`, що дає змогу серверу не блокуватися під час очікування відповіді від бази.

Отримання списку книг. Маршрут `GET /api/books` повертає книги з урахуванням пошукового запиту, обраної категорії, способу сортування та номера сторінки. Пошук виконується одночасно за назвою, описом і категорією за допомогою оператора `$or` та регулярного виразу; пагінацію реалізовано методами `skip` і `limit`, тож за один запит повертається лише потрібна сторінка каталогу разом із загальною кількістю книг.

Завантаження каталогу на клієнті. Головний компонент App за допомогою хука `useEffect` надсилає `axios`-запит до сервера й зберігає отримані книги у стані компонента. Будь-яка зміна пошукового рядка, категорії, сортування чи номера сторінки автоматично спричиняє повторний запит і оновлення каталогу. Загальний вигляд каталогу зі списком книг та посторінковою навігацією показано на рисунку 2.4 вище.

Додавання книги. Натискання кнопки «Додати книгу» відкриває модальне вікно з порожньою формою (рисунки 2.6). Користувач заповнює поля та обирає категорію (рисунки 2.7), після чого клієнт надсилає на сервер запит `POST`, сервер створює новий документ у базі, і каталог оновлюється.

					2026.KP.122.321.10.00.00 ПЗ	Арк.
						16
Зм.	Арк.	№ докум.	Підпис	Дата		

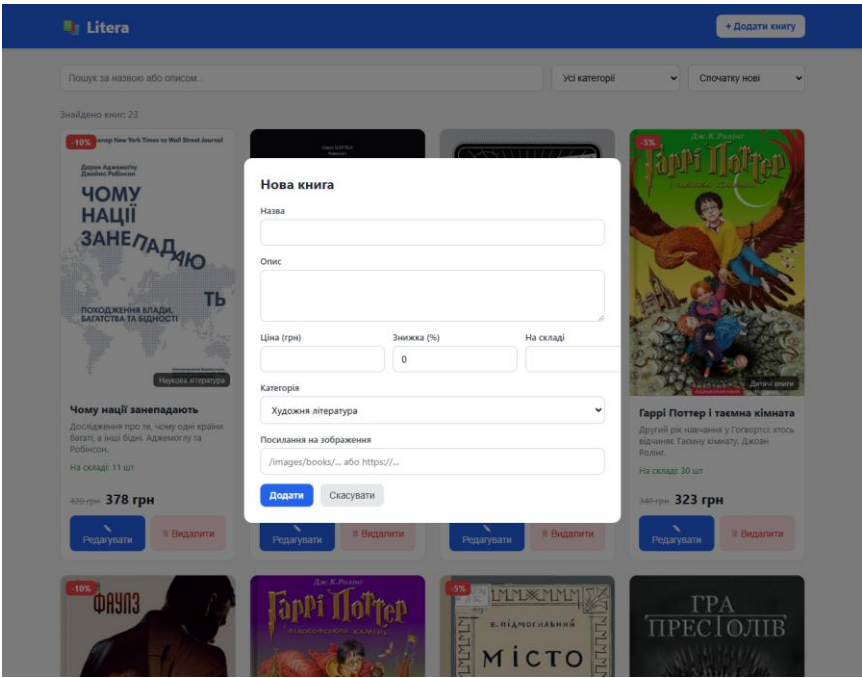


Рисунок 2.6 – Модальне вікно додавання книги

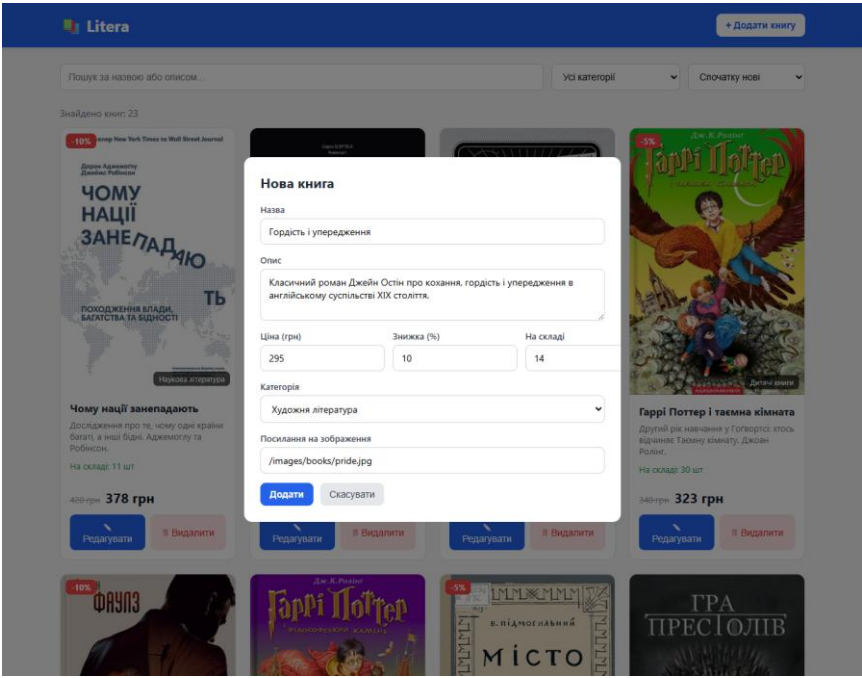


Рисунок 2.7 – Заповнена форма нової книги

Перевірка введених даних. Дані перевіряються двічі. Спершу спрацьовує клієнтська валідація у компоненті форми: назва й опис не можуть бути порожніми чи надто короткими, ціна має бути додатною, знижка – у межах від 0 до 100 %, а кількість – невід’ємною. Якщо виявлено помилку, під відповідним полем з’являється підказка, а запит на сервер не надсилається (рисунок 2.8). Додатково ті самі правила перевіряє схема Mongoose уже на сервері, що захищає базу даних від некоректних записів.

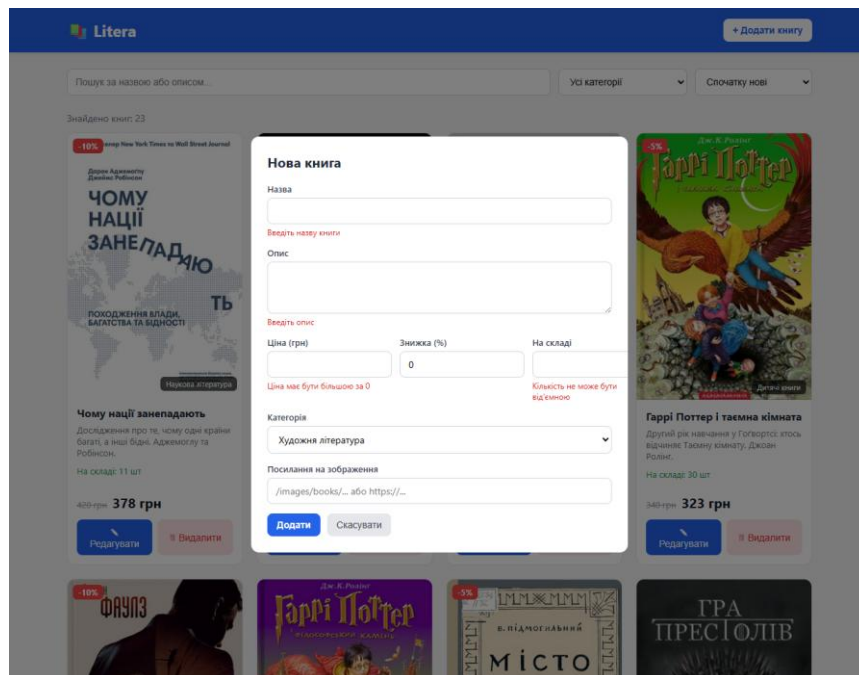


Рисунок 2.8 – Повідомлення про помилки під час валідації

Редагування книги. Кнопка «Редагувати» на картці відкриває те саме вікно, але вже заповнене даними обраної книги (рисунок 2.9). Після внесення змін клієнт надсилає запит PUT із ідентифікатором книги, і відповідний документ у базі оновлюється.

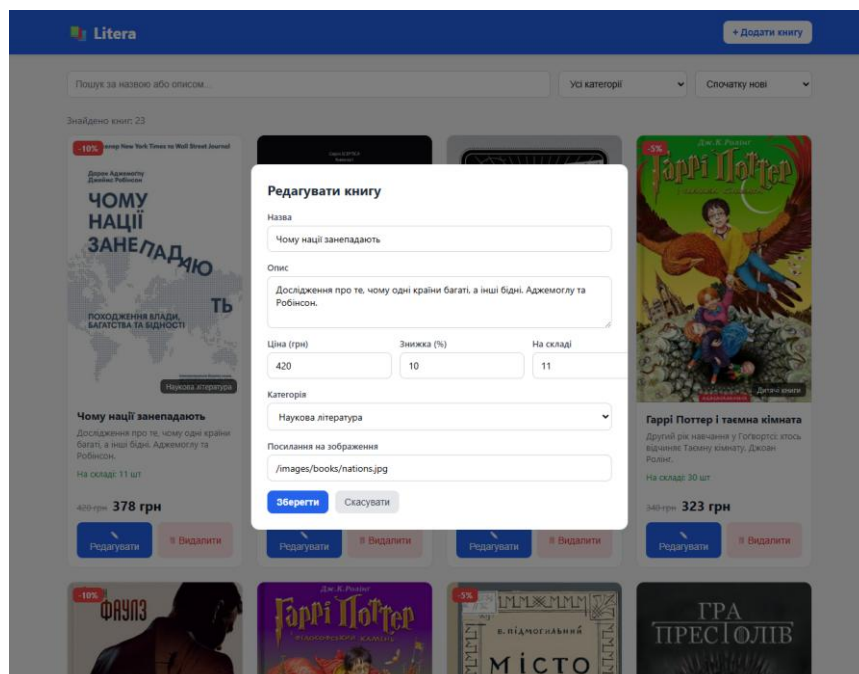


Рисунок 2.9 – Редагування характеристик книги

Видалення книги. Кнопка «Видалити» викликає окреме вікно з підтвердженням (рисунок 2.10). Лише після підтвердження клієнт надсилає

запит DELETE, і книга вилучається з бази, що убезпечує від випадкового видалення.

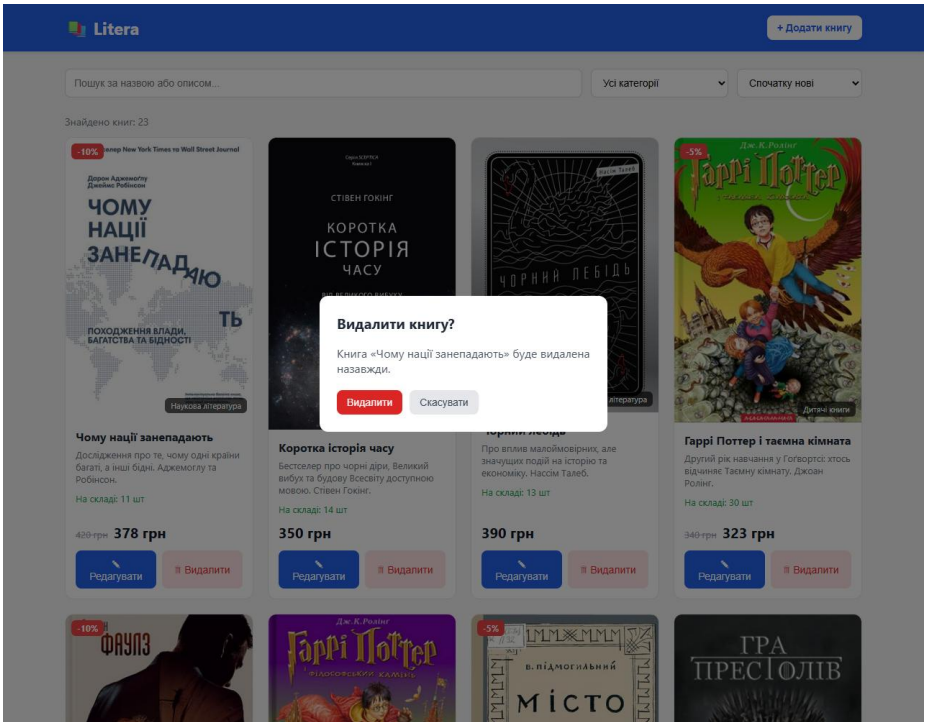


Рисунок 2.10 – Підтвердження видалення книги

Пошук, фільтр та сортування. Поле пошуку знаходить книги за збігом у назві, описі або категорії (рисунок 2.11). Випадний список категорій залишає на екрані лише книги обраного розділу (рисунок 2.12), а список сортування впорядковує їх за ціною чи за назвою. Усі ці параметри передаються на сервер, і той повертає вже відфільтрований та впорядкований список.

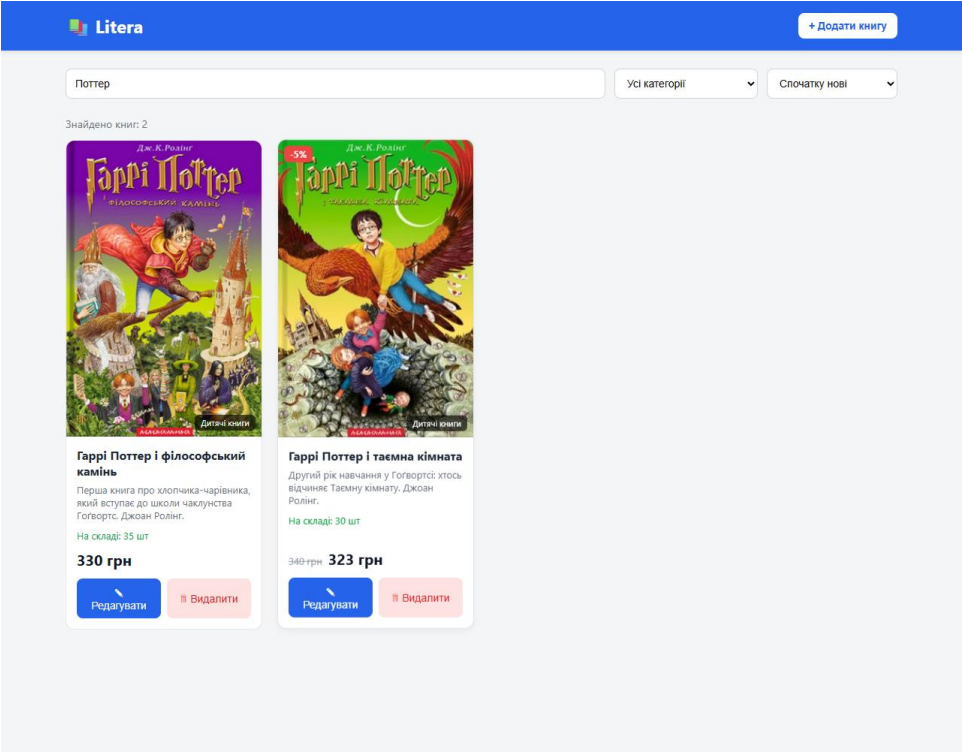


Рисунок 2.11 – Пошук книги за назвою

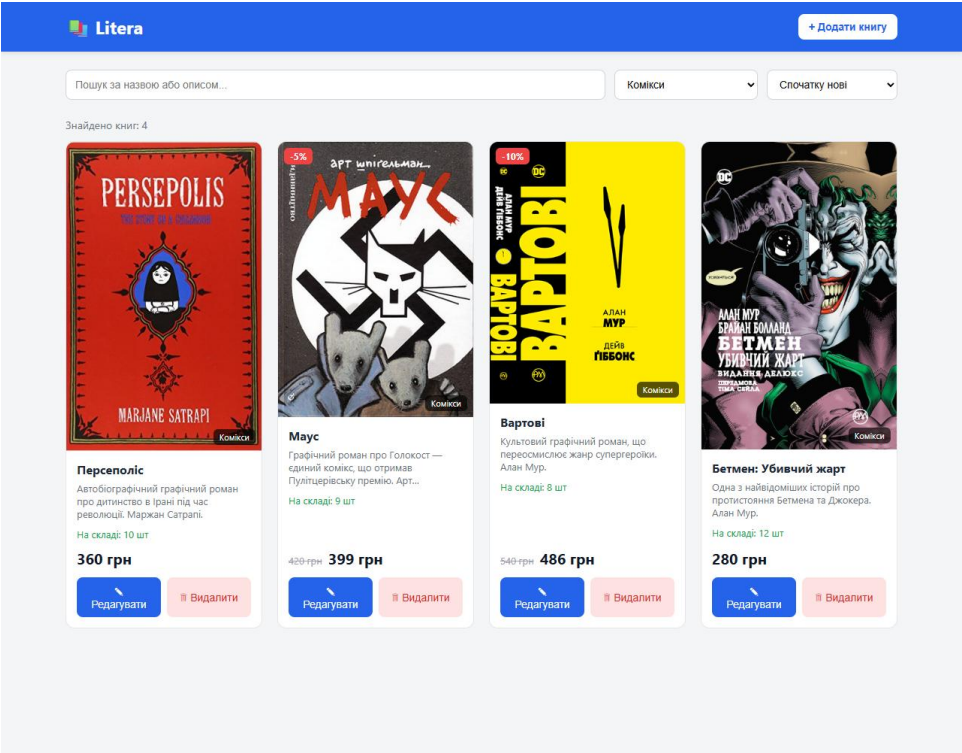


Рисунок 2.12 – Фільтрування книг за категорією «Комікси»

Узагальнений алгоритм збереження книги, що поєднує клієнтську валідацію, звернення до сервера й оновлення каталогу, подано у вигляді блок-схеми на рисунку 2.13.

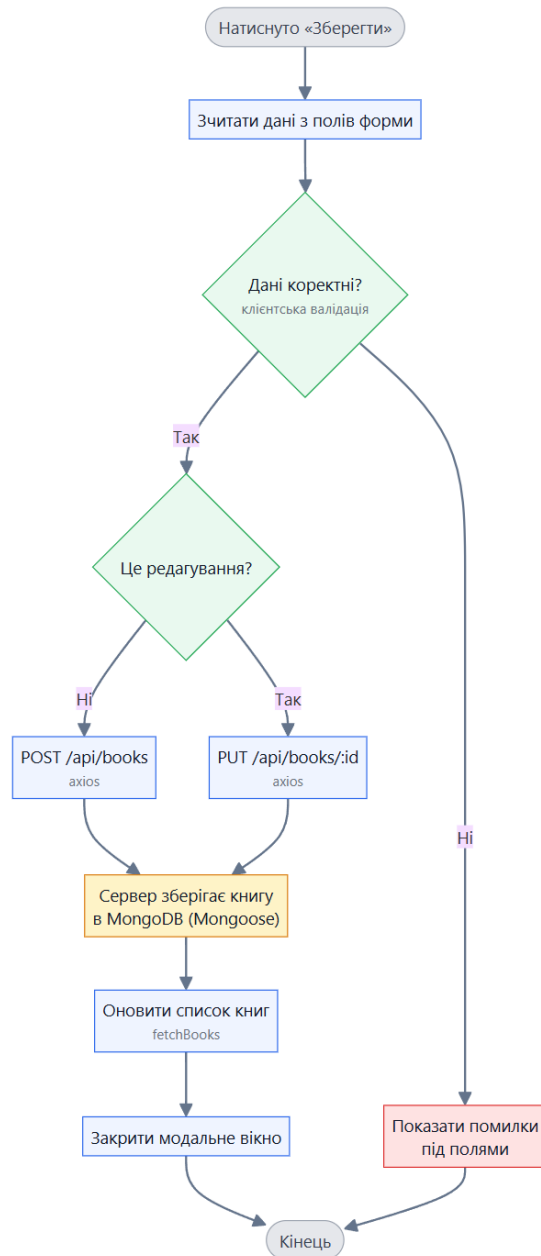


Рисунок 2.13 – Алгоритм збереження книги з перевіркою даних

2.5 Тестування веб-сайту

Випробування додатку проведено методом функціонального тестування: для кожної дії перевіряли відповідність отриманого результату очікуваному. Перевірено всі основні можливості; результати зведено в таблиці 2.1.

№	Перевірювана дія	Очікуваний результат	Підсумок
1	Додавання книги з коректними даними	Книга з'являється в каталозі	Відповідає
2	Спроба зберегти порожню форму	Виводяться повідомлення про помилки	Відповідає
3	Редагування характеристик книги	Дані книги оновлюються	Відповідає
4	Видалення книги з підтвердженням	Книгу вилучено з каталогу	Відповідає
5	Пошук за назвою	Показано лише відповідні книги	Відповідає
6	Фільтрування за категорією	Показано книги лише обраної категорії	Відповідає
7	Сортування за ціною	Книги шикуються за зростанням ціни	Відповідає
8	Перехід між сторінками каталогу	Виводиться наступна сторінка книг	Відповідає

Таблиця 2.1 – Результати тестування веб-додатку

Адаптивність перевірено на екранах різної ширини: на широкому моніторі картки розташовуються в чотири стовпці, на планшеті – у два, на смартфоні – в один. Перевірку сумісності виконано у браузерях Google Chrome, Microsoft Edge та Mozilla Firefox – додаток працює в них однаково. Усі заплановані функції відпрацьовують правильно, помилок не виявлено.

3 СПЕЦІАЛЬНИЙ РОЗДІЛ

3.1 Інструкція з розміщення сайту в мережі Інтернет

Оскільки додаток складається з двох частин, його розміщення в мережі Інтернет потребує трьох складових: бази даних, серверної частини та клієнтської частини. Базу даних зручно розмістити у хмарному сервісі MongoDB Atlas, який надає безкоштовний обліковий запис; адреса підключення до бази зберігається в налаштуваннях сервера у змінній середовища.

Послідовність розміщення додатку така:

1. створити безкоштовний кластер бази даних у сервісі MongoDB Atlas та отримати рядок підключення;
2. розгорнути серверну частину (папку backend) на хостингу, що підтримує Node.js (наприклад, Render або Railway), вказавши рядок підключення до бази у змінній середовища;
3. зібрати клієнтську частину командою `npm run build` і розмістити отримані статичні файли на хостингу для статичних сайтів або віддавати їх тим самим сервером Express;
4. у налаштуваннях клієнта вказати адресу серверного API, після чого додаток стає доступним користувачам за отриманим посиланням.

Для локального запуску достатньо встановити залежності командою `npm install` в обох папках, заповнити базу книгами командою `npm run seed` та запустити серверну і клієнтську частини командою `npm run dev`.

3.2 Інструкція з обслуговування та наповнення сайту

Супровід веб-додатку зводиться переважно до наповнення каталогу книгами та підтримання серверної частини й бази даних у робочому стані. Усі дії з каталогом виконуються безпосередньо на сторінці без редагування коду.

Додавання книги: натиснути кнопку «Додати книгу», заповнити поля форми, за потреби вказати посилання на обкладинку та натиснути «Додати».

					2026.KP.122.321.10.00.00 ПЗ	Арк.
						23
Зм.	Арк.	№ докум.	Підпис	Дата		

Редагування: натиснути кнопку «Редагувати» на потрібній картці, змінити значення полів і зберегти зміни.

Видалення: натиснути кнопку «Видалити» та підтвердити дію у вікні, що з'явиться.

Резервне копіювання та відновлення: сервіс MongoDB Atlas виконує автоматичне резервне копіювання бази даних; за потреби початковий набір книг можна відновити, повторно запустивши скрипт наповнення `npm run seed`.

3.3 Інструкція з популяризації та підтримки сайту

Щоб веб-додаток був помітним для відвідувачів, варто подбати про його просування та подальший супровід. Навіть для невеликого навчального проєкту корисно дотримуватися кількох простих рекомендацій.

Пошукова оптимізація. У розмітці сторінки слід зазначити змістовний заголовок та опис у метатеггах, дібрати ключові слова, пов'язані з продажем книг, і вказати зрозумілі назви зображень. Це підвищує позиції сторінки у видачі пошукових систем.

Поширення в соціальних мережах. Посилання на додаток можна розмістити у профілях і тематичних спільнотах, присвячених читанню, що приваблює нових відвідувачів без витрат на рекламу.

Підтримка та оновлення. Каталог потрібно регулярно поповнювати новинками та вилучати застарілі позиції, стежити за справністю серверної частини й посилань на обкладинки, а також за відгуками користувачів удосконалювати інтерфейс. Завдяки збереженню коду в репозиторії будь-яке оновлення легко публікується повторно.

Дотримання цих рекомендацій дає змогу поступово розширювати аудиторію додатку та підтримувати його в робочому й привабливому стані.

					2026.KP.122.321.10.00.00 ПЗ	Арк.
						24
Зм.	Арк.	№ докум.	Підпис	Дата		

ВИСНОВКИ

У результаті виконання курсової роботи створено клієнт-серверний веб-додаток книжкового онлайн-магазину «Літера» на основі стеку технологій MERN. Поставлену мету досягнуто, а всі визначені завдання виконано. Зокрема, у ході роботи:

1. оглянуто й порівняно наявні книжкові інтернет-магазини (Yakaboo, Книгарня «Є», Rozetka) та визначено типовий набір їхніх можливостей;
2. складено технічне завдання й сформульовано функціональні вимоги до веб-додатку;
3. обрано технології реалізації – React і Vite для клієнтської частини, Node.js та Express для серверної, MongoDB і Mongoose для зберігання даних;
4. спроектовано структуру бази даних та REST API для роботи з каталогом книг;
5. розроблено інтерфейс користувача у вигляді односторінкового React-додатку з картками книг і модальними вікнами;
6. запрограмовано перегляд, пошук, фільтрування, сортування, пагінацію, додавання, редагування та видалення книг із перевіркою даних на клієнті й на сервері;
7. проведено тестування додатку та підготовлено інструкції з його розміщення, обслуговування й популяризації.

Практична цінність роботи полягає у створенні завершеного клієнт-серверного веб-додатку з реальною базою даних, який демонструє основні можливості каталогу книжкового інтернет-магазину. Розроблений додаток може слугувати основою для подальшого вдосконалення – додавання кошика, особистих кабінетів, оформлення замовлень та системи авторизації.

					2026.KP.122.321.10.00.00 ПЗ	Арк.
						25
Зм.	Арк.	№ докум.	Підпис	Дата		

ПЕРЕЛІК ПОСИЛАНЬ

1. Онлайн-магазин Yakaboo [Електронний ресурс]. – Режим доступу: <https://www.yakaboo.ua/>.
2. Книгарня «Є» [Електронний ресурс]. – Режим доступу: <https://book-ye.com.ua/>.
3. JavaScript [Електронний ресурс]. – Режим доступу: <https://developer.mozilla.org/uk/docs/Web/JavaScript>.
4. React – A JavaScript library for building user interfaces [Електронний ресурс]. – Режим доступу: <https://react.dev/>.
5. Vite – Next Generation Frontend Tooling [Електронний ресурс]. – Режим доступу: <https://vitejs.dev/>.
6. Node.js Documentation [Електронний ресурс]. – Режим доступу: <https://nodejs.org/uk/docs>.
7. Express – Node.js web application framework [Електронний ресурс]. – Режим доступу: <https://expressjs.com/>.
8. MongoDB Documentation [Електронний ресурс]. – Режим доступу: <https://www.mongodb.com/docs/>.
9. Mongoose ODM Documentation [Електронний ресурс]. – Режим доступу: <https://mongoosejs.com/docs/>.
10. Axios – Promise based HTTP client [Електронний ресурс]. – Режим доступу: <https://axios-http.com/>.

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						26
Зм.	Арк.	№ докум.	Підпис	Дата		

ДОДАТОК А

Лістинг серверної частини (backend)

server.js

```
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import connectDB from './config/db.js';
import bookRoutes from './routes/bookRoutes.js';

dotenv.config();

const app = express();

connectDB();

// Інакше фронтенд (він на іншому порту) не дістанеться до сервера
app.use(cors({ origin: /^http:\/\/localhost:\d+$/ }));

app.use(express.json());

app.use('/api/books', bookRoutes);

// Перевірка, що сервер живий
app.get('/', (req, res) => {
  res.json({ message: '📖 Litera API працює!', books: '/api/books' });
});

const PORT = process.env.PORT || 5001;
app.listen(PORT, () => {
  console.log(`Сервер запущено на порті ${PORT}`);
});
```

config/db.js

```
import mongoose from 'mongoose';

// Підключаюся до MongoDB (адреса бази - у .env)
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URI);
    console.log(`MongoDB підключено: ${conn.connection.host}`);
  } catch (error) {
    // Не достукався до бази - далі немає сенсу, виходжу
    console.error(`Помилка підключення до MongoDB: ${error.message}`);
    process.exit(1);
  }
};

export default connectDB;
```

models/Book.js

```
import mongoose from 'mongoose';

// Категорії - фіксований список (такий самий у фільтрі на фронтенді)
export const CATEGORIES = [
  'Художня література',
  'Наукова література',
  'Дитячі книги',
];
```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						27
Зм.	Арк.	№ докум.	Підпис	Дата		

```

    'Комікси',
    'Навчальна література',
  ];

  // Схема книги: 7 полів за завданням. Тут же перевірки, щоб у базу не потрапило сміття
  const bookSchema = new mongoose.Schema(
    {
      title: {
        type: String,
        required: [true, "Назва є обов'язковою"],
        trim: true,
      },
      description: {
        type: String,
        required: [true, "Опис є обов'язковим"],
        trim: true,
      },
      price: {
        type: Number,
        required: [true, "Ціна є обов'язковою"],
        min: [0, "Ціна не може бути від'ємною"],
      },
      discount: {
        type: Number,
        default: 0,
        min: [0, "Знижка не може бути від'ємною"],
        max: [100, 'Знижка не може перевищувати 100%'],
      },
      stock: {
        type: Number,
        required: [true, 'Кількість на складі є обов\'язковою'],
        min: [0, "Кількість не може бути від'ємною"],
        default: 0,
      },
      image: {
        type: String,
        default: '',
      },
      category: {
        type: String,
        required: [true, "Категорія є обов'язковою"],
        enum: {
          values: CATEGORIES,
          message: 'Недопустима категорія',
        },
      },
    },
    { timestamps: true }
  );

  export default mongoose.model('Book', bookSchema);

```

routes/bookRoutes.js

```

import express from 'express';
import {
  getBooks,
  getBookById,
  createBook,
  updateBook,
  deleteBook,
} from '../controllers/bookController.js';

const router = express.Router();

router.get('/', getBooks);
router.get('/:id', getBookById);
router.post('/', createBook);

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						28
Зм.	Арк.	№ докум.	Підпис	Дата		

```

router.put('/:id', updateBook);
router.delete('/:id', deleteBook);

export default router;

```

controllers/bookController.js

```

import Book from '../models/Book.js';

// Значення мають збігатися з селектом сортування на фронтенді
const SORT_OPTIONS = {
  newest: { createdAt: -1 },
  price_asc: { price: 1 },
  price_desc: { price: -1 },
  title_asc: { title: 1 },
  title_desc: { title: -1 },
};

// GET /api/books - список книг: пошук, фільтр, сортування, пагінація
export const getBooks = async (req, res) => {
  try {
    const page = parseInt(req.query.page) || 1;
    const limit = parseInt(req.query.limit) || 8;
    const search = (req.query.search || '').trim();
    const category = req.query.category || '';
    const sort = SORT_OPTIONS[req.query.sort] || SORT_OPTIONS.newest;

    const filter = {};

    // Шукаю одразу по назві, опису й категорії (і - без урахування регістру)
    if (search) {
      const regex = { $regex: search, $options: 'i' };
      filter.$or = [{ title: regex }, { description: regex }, { category: regex }];
    }

    if (category) {
      filter.category = category;
    }

    const total = await Book.countDocuments(filter);
    const pages = Math.ceil(total / limit) || 1;

    // skip + limit - оце і є пагінація
    const books = await Book.find(filter)
      .sort(sort)
      .skip((page - 1) * limit)
      .limit(limit);

    res.json({ books, page, pages, total });
  } catch (error) {
    res.status(500).json({ message: 'Помилка отримання книг', error: error.message });
  }
};

// GET /api/books/:id - одна книга
export const getBookById = async (req, res) => {
  try {
    const book = await Book.findById(req.params.id);
    if (!book) return res.status(404).json({ message: 'Книгу не знайдено' });
    res.json(book);
  } catch (error) {
    res.status(500).json({ message: 'Помилка отримання книги', error: error.message });
  }
};

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						29
Зм.	Арк.	№ докум.	Підпис	Дата		

```

// POST /api/books - створити книгу
export const createBook = async (req, res) => {
  try {
    const { title, description, price, discount, stock, image, category } = req.body;
    const book = await Book.create({ title, description, price, discount, stock,
image, category });
    res.status(201).json(book);
  } catch (error) {
    // 400 - якщо схема не пропустила дані
    res.status(400).json({ message: error.message });
  }
};

// PUT /api/books/:id - оновити книгу
export const updateBook = async (req, res) => {
  try {
    const { title, description, price, discount, stock, image, category } = req.body;
    // new: true - повертає оновлену книгу; runValidators - щоб перевірки спрацювали й
при оновленні
    const book = await Book.findByIdAndUpdate(
      req.params.id,
      { title, description, price, discount, stock, image, category },
      { new: true, runValidators: true }
    );
    if (!book) return res.status(404).json({ message: 'Книгу не знайдено' });
    res.json(book);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};

// DELETE /api/books/:id - видалити книгу
export const deleteBook = async (req, res) => {
  try {
    const book = await Book.findByIdAndDelete(req.params.id);
    if (!book) return res.status(404).json({ message: 'Книгу не знайдено' });
    res.json({ message: 'Книгу видалено' });
  } catch (error) {
    res.status(500).json({ message: 'Помилка видалення книги', error: error.message
});
  }
};

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		30

ДОДАТОК Б

Лістинг клієнтської частини (frontend)

src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './App.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

src/api.js

```
import axios from 'axios';

// Один axios на весь застосунок, щоб не писати повну адресу щоразу
const api = axios.create({
  baseURL: 'http://localhost:5001/api',
});

export default api;
```

src/App.jsx

```
import { useState, useEffect, useCallback } from 'react';
import api from './api';
import BookCard from './components/BookCard';
import BookFormModal from './components/BookFormModal';
import ConfirmModal from './components/ConfirmModal';

const CATEGORIES = [
  'Художня література',
  'Наукова література',
  'Дитячі книги',
  'Комікси',
  'Навчальна література',
];

const SORT_OPTIONS = [
  { value: 'newest', label: 'Спочатку нові' },
  { value: 'price_asc', label: 'Ціна: від дешевших' },
  { value: 'price_desc', label: 'Ціна: від дорожчих' },
  { value: 'title_asc', label: 'Назва: А-Я' },
  { value: 'title_desc', label: 'Назва: Я-А' },
];

function App() {
  const [books, setBooks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  // Те, що користувач задає на панелі: пошук, категорія, сортування і номер сторінки
  const [search, setSearch] = useState('');
  const [category, setCategory] = useState('');
  const [sort, setSort] = useState('newest');
  const [page, setPage] = useState(1);
  const [pages, setPages] = useState(1);
  const [total, setTotal] = useState(0);
```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		31

```

// Яке з модальних вікон зараз відкрите
const [showForm, setShowForm] = useState(false);
const [editingBook, setEditingBook] = useState(null);
const [deletingBook, setDeletingBook] = useState(null);

// Тягну книги з сервера; useCallback - щоб функція не створювалась наново щоразу
const fetchBooks = useCallback(async () => {
  setLoading(true);
  setError('');
  try {
    const { data } = await api.get('/books', {
      params: { search, category, sort, page, limit: 8 },
    });
    setBooks(data.books);
    setPages(data.pages);
    setTotal(data.total);
  } catch (err) {
    setError('Не вдалося завантажити книги. Перевірте, чи запущено сервер.');
```

finally {

```

    setLoading(false);
  }
}, [search, category, sort, page]);

useEffect(() => {
  fetchBooks();
}, [fetchBooks]);

// Якщо змінено пошук/категорію/сортування - логічно починаю знову з 1-ї сторінки
useEffect(() => {
  setPage(1);
}, [search, category, sort]);

// Зберігаю книгу: якщо є editingBook - оновлюю (PUT), інакше створюю (POST)
const handleSave = async (bookData) => {
  try {
    if (editingBook) {
      await api.put(`/books/${editingBook._id}`, bookData);
    } else {
      await api.post('/books', bookData);
    }
    setShowForm(false);
    setEditingBook(null);
    fetchBooks();
  } catch (err) {
    alert(err.response?.data?.message || 'Помилка збереження');
  }
};

// Видаляю книгу (після підтвердження у вікні)
const handleDelete = async () => {
  try {
    await api.delete(`/books/${deletingBook._id}`);
    setDeletingBook(null);
    // Якщо це була остання книга на сторінці, повертаюся на сторінку назад
    if (books.length === 1 && page > 1) setPage(page - 1);
    else fetchBooks();
  } catch (err) {
    alert('Помилка видалення');
  }
};

const openCreate = () => {
  setEditingBook(null);
  setShowForm(true);
};

const openEdit = (book) => {

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						32
Зм.	Арк.	№ докум.	Підпис	Дата		


```

    setEditingBook(book);
    setShowForm(true);
  };

  return (
    <div className="app">
      <header className="header">
        <div className="container header-inner">
          <h1 className="logo"><img alt="Litera logo" data-bbox="425 170 445 185"/> Litera</h1>
          <button className="btn btn-primary" onClick={openCreate}>
            + Додати книгу
          </button>
        </div>
      </header>

      <main className="container">
        {/* Панель: пошук, фільтр категорії, сортування */}
        <div className="toolbar">
          <input
            className="toolbar-search"
            type="text"
            placeholder="Пошук за назвою або описом..."
            value={search}
            onChange={(e) => setSearch(e.target.value)}
          />

          <select value={category} onChange={(e) => setCategory(e.target.value)}>
            <option value="">Усі категорії</option>
            {CATEGORIES.map((c) => (
              <option key={c} value={c}>{c}</option>
            ))}
          </select>

          <select value={sort} onChange={(e) => setSort(e.target.value)}>
            {SORT_OPTIONS.map((o) => (
              <option key={o.value} value={o.value}>{o.label}</option>
            ))}
          </select>
        </div>

        {loading && <p className="info">Завантаження...</p>}
        {error && <p className="info error">{error}</p>}}

        {!loading && !error && books.length === 0 && (
          <p className="info">Книг не знайдено.</p>
        )}

        {!loading && !error && books.length > 0 && (
          <>
            <p className="result-count">Знайдено книг: {total}</p>

            <div className="grid">
              {books.map((book) => (
                <BookCard
                  key={book._id}
                  book={book}
                  onEdit={openEdit}
                  onDelete={setDeletingBook}
                />
              ))}
            </div>

            {/* Пагінація */}
            {pages > 1 && (
              <div className="pagination">
                <button
                  className="btn btn-secondary"
                  disabled={page === 1}

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						33
Зм.	Арк.	№ докум.	Підпис	Дата		

```

        onClick={() => setPage(page - 1)}
      >
        ← Назад
      </button>
      <span className="page-info">
        Сторінка {page} з {pages}
      </span>
      <button
        className="btn btn-secondary"
        disabled={page === pages}
        onClick={() => setPage(page + 1)}
      >
        Далі →
      </button>
    </div>
  )}
</>
)}
</main>

<BookFormModal
  isOpen={showForm}
  editingBook={editingBook}
  onSubmit={handleSave}
  onCancel={() => {
    setShowForm(false);
    setEditingBook(null);
  }}
/>

<ConfirmModal
  isOpen={deletingBook !== null}
  title="Видалити книгу?"
  message={`Книга «${deletingBook?.title}» буде видалена назавжди.`}
  onConfirm={handleDelete}
  onCancel={() => setDeletingBook(null)}
/>
</div>
);
}

export default App;

```

src/components/BookCard.jsx

```

// Картка однієї книги
function BookCard({ book, onEdit, onDelete }) {
  // Рахую ціну зі знижкою (якщо вона є)
  const finalPrice =
    book.discount > 0
      ? Math.round(book.price * (1 - book.discount / 100))
      : book.price;

  return (
    <div className="card">
      <div className="card-image">
        <img
          src={book.image || 'https://placeholder.co/300x450?text=Без+фото'}
          alt={book.title}
        />
        {book.discount > 0 && <span className="badge-discount">-
{book.discount}%</span>}
        <span className="badge-category">{book.category}</span>
      </div>

      <div className="card-body">
        <h3 className="card-title">{book.title}</h3>
        <p className="card-desc">{book.description}</p>
      </div>
    </div>
  );
}

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						34
Зм.	Арк.	№ докум.	Підпис	Дата		

```

    <ul className="card-props">
      <li>
        <span className={book.stock > 0 ? 'in-stock' : 'out-stock'}>
          {book.stock > 0 ? `На складі: ${book.stock} шт` : 'Немає в наявності'}
        </span>
      </li>
    </ul>

    <div className="card-price">
      {book.discount > 0 && <span className="old-price">{book.price} грн</span>}
      <span className="final-price">{finalPrice} грн</span>
    </div>

    <div className="card-actions">
      <button className="btn btn-edit" onClick={() => onEdit(book)}>
        ✎ Редагувати
      </button>
      <button className="btn btn-delete" onClick={() => onDelete(book)}>
        🗑 Видалити
      </button>
    </div>
  </div>
);
}

export default BookCard;

```

src/components/BookFormModal.jsx

```

import { useState, useEffect } from 'react';

const CATEGORIES = [
  'Художня література',
  'Наукова література',
  'Дитячі книги',
  'Комікси',
  'Навчальна література',
];

const emptyForm = {
  title: '',
  description: '',
  price: '',
  discount: '0',
  stock: '',
  image: '',
  category: CATEGORIES[0],
};

// Одне вікно і на додавання, і на редагування (відрізняю по editingBook)
function BookFormModal({ isOpen, editingBook, onSubmit, onCancel }) {
  const [form, setForm] = useState(emptyForm);
  const [errors, setErrors] = useState({});

  // Коли редагую - підставляю дані книги, коли додаю - порожня форма
  useEffect(() => {
    if (editingBook) {
      setForm({
        title: editingBook.title,
        description: editingBook.description,
        price: String(editingBook.price),
        discount: String(editingBook.discount),
        stock: String(editingBook.stock),
        image: editingBook.image || '',
      });
    }
  }, [editingBook]);

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		35

```

        category: editingBook.category,
    });
    } else {
        setForm(emptyForm);
    }
    setErrors({});
}, [editingBook, isOpen]);

if (!isOpen) return null;

const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
};

// Перевіряю поля перед відправкою; повертає true, якщо все добре
const validate = () => {
    const e = {};
    if (!form.title.trim()) e.title = 'Введіть назву книги';
    else if (form.title.trim().length < 2) e.title = 'Назва занадто коротка';

    if (!form.description.trim()) e.description = 'Введіть опис';
    else if (form.description.trim().length < 10) e.description = 'Опис має містити
щонайменше 10 символів';

    if (form.price === '' || Number(form.price) <= 0) e.price = 'Ціна має бути більшою
за 0';

    const discount = Number(form.discount);
    if (form.discount === '' || discount < 0 || discount > 100) e.discount = 'Знижка
від 0 до 100';

    if (form.stock === '' || Number(form.stock) < 0) e.stock = "Кількість не може бути
від'ємною";

    setErrors(e);
    return Object.keys(e).length === 0;
};

const handleSubmit = (event) => {
    event.preventDefault();
    if (!validate()) return;

    onSubmit({
        title: form.title.trim(),
        description: form.description.trim(),
        price: Number(form.price),
        discount: Number(form.discount),
        stock: Number(form.stock),
        image: form.image.trim(),
        category: form.category,
    });
};

return (
    <div className="modal-overlay" onClick={onCancel}>
        <div className="modal" onClick={e => e.stopPropagation()}>
            <h2 className="modal-title">
                {editingBook ? 'Редагувати книгу' : 'Нова книга'}
            </h2>

            <form onSubmit={handleSubmit} noValidate>
                <div className="form-group">
                    <label>Назва</label>
                    <input name="title" value={form.title} onChange={handleChange} />
                    {errors.title && <span className="error">{errors.title}</span>}
                </div>

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						36
Зм.	Арк.	№ докум.	Підпис	Дата		

```

        <div className="form-group">
            <label>Опис</label>
            <textarea name="description" rows={3} value={form.description}
onChange={handleChange} />
            {errors.description && <span
className="error">{errors.description}</span>}
        </div>

        <div className="form-row">
            <div className="form-group">
                <label>Ціна (грн)</label>
                <input type="number" name="price" value={form.price}
onChange={handleChange} />
                {errors.price && <span className="error">{errors.price}</span>}
            </div>
            <div className="form-group">
                <label>Знижка (%)</label>
                <input type="number" name="discount" value={form.discount}
onChange={handleChange} />
                {errors.discount && <span className="error">{errors.discount}</span>}
            </div>
            <div className="form-group">
                <label>На складі</label>
                <input type="number" name="stock" value={form.stock}
onChange={handleChange} />
                {errors.stock && <span className="error">{errors.stock}</span>}
            </div>
        </div>

        <div className="form-group">
            <label>Категорія</label>
            <select name="category" value={form.category} onChange={handleChange}>
                {CATEGORIES.map((c) => (
                    <option key={c} value={c}>{c}</option>
                ))}
            </select>
        </div>

        <div className="form-group">
            <label>Посилання на зображення</label>
            <input name="image" value={form.image} onChange={handleChange}
placeholder="/images/books/... або https://..." />
        </div>

        <div className="modal-actions">
            <button type="submit" className="btn btn-primary">
                {editingBook ? 'Зберегти' : 'Додати'}
            </button>
            <button type="button" className="btn btn-secondary" onClick={onCancel}>
                Скасувати
            </button>
        </div>
    </form>
</div>
</div>
);
}

export default BookFormModal;

```

src/components/ConfirmModal.jsx

```

// Вікно підтвердження перед видаленням книги
function ConfirmModal({ isOpen, title, message, onConfirm, onCancel }) {
    if (!isOpen) return null;

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						37
Зм.	Арк.	№ докум.	Підпис	Дата		

```

return (
  <div className="modal-overlay" onClick={onCancel}>
    <div className="modal modal-small" onClick={(e) => e.stopPropagation()}>
      <h2 className="modal-title">{title}</h2>
      <p className="confirm-text">{message}</p>
      <div className="modal-actions">
        <button className="btn btn-danger" onClick={onConfirm}>
          Видалити
        </button>
        <button className="btn btn-secondary" onClick={onCancel}>
          Скасувати
        </button>
      </div>
    </div>
  </div>
);
}

export default ConfirmModal;

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		38

ДОДАТОК В

Лістинг файлу стилів App.css

src/App.css

```
/* Скидання стилів браузера */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Arial, sans-serif;
  background: #f4f5f7;
  color: #1f2937;
}

.container {
  max-width: 1140px;
  margin: 0 auto;
  padding: 0 16px;
}

/* Шапка сайту */
.header {
  background: #2563eb;
  color: #fff;
  padding: 16px 0;
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
}

.header-inner {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.logo {
  font-size: 24px;
}

/* Панель пошуку і фільтрів */
.toolbar {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  margin: 24px 0;
}

.toolbar-search {
  flex: 1;
  min-width: 220px;
}

.toolbar input,
.toolbar select {
  padding: 10px 12px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
  font-size: 14px;
  background: #fff;
}

.result-count {
  margin-bottom: 12px;
  color: #6b7280;
  font-size: 14px;
}
```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						39
Зм.	Арк.	№ докум.	Підпис	Дата		

```

}

.info {
  text-align: center;
  padding: 40px 0;
  color: #6b7280;
}

.info.error {
  color: #dc2626;
}

/* Сітка карток (grid) */
.grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 20px;
}

@media (max-width: 900px) {
  .grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (max-width: 520px) {
  .grid {
    grid-template-columns: 1fr;
  }
}

/* Картка книги */
.card {
  background: #fff;
  border: 1px solid #e5e7eb;
  border-radius: 12px;
  overflow: hidden;
  display: flex;
  flex-direction: column;
  transition: box-shadow 0.2s, transform 0.2s;
}

.card:hover {
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.1);
  transform: translateY(-3px);
}

.card-image {
  position: relative;
  aspect-ratio: 3 / 4;
  background: #f3f4f6;
}

.card-image img {
  width: 100%;
  height: 100%;
  object-fit: cover;
}

.badge-discount {
  position: absolute;
  top: 8px;
  left: 8px;
  background: #ef4444;
  color: #fff;
  font-size: 12px;
  font-weight: 700;
  padding: 3px 8px;
  border-radius: 6px;
}

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						40
Зм.	Арк.	№ докум.	Підпис	Дата		


```

.badge-category {
  position: absolute;
  bottom: 8px;
  right: 8px;
  background: rgba(0, 0, 0, 0.65);
  color: #fff;
  font-size: 11px;
  padding: 3px 8px;
  border-radius: 6px;
}

.card-body {
  padding: 14px;
  display: flex;
  flex-direction: column;
  flex: 1;
}

.card-title {
  font-size: 16px;
  margin-bottom: 6px;
}

.card-desc {
  font-size: 13px;
  color: #6b7280;
  margin-bottom: 10px;
  display: -webkit-box;
  -webkit-line-clamp: 3;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

.card-props {
  list-style: none;
  font-size: 13px;
  margin-bottom: 10px;
}

.in-stock {
  color: #16a34a;
}

.out-stock {
  color: #dc2626;
}

.card-price {
  margin-top: auto;
  margin-bottom: 12px;
}

.old-price {
  text-decoration: line-through;
  color: #9ca3af;
  font-size: 13px;
  margin-right: 8px;
}

.final-price {
  font-size: 20px;
  font-weight: 700;
  color: #111827;
}

.card-actions {
  display: flex;
  gap: 8px;
}

/* КНОПКИ */

```

					2026.KP.122.321.10.00.00 ПЗ	Арк.
						41
Зм.	Арк.	№ докум.	Підпис	Дата		

```

.btn {
  border: none;
  border-radius: 8px;
  padding: 9px 14px;
  font-size: 14px;
  cursor: pointer;
  transition: background 0.15s, opacity 0.15s;
}

.btn:disabled {
  opacity: 0.4;
  cursor: not-allowed;
}

.btn-primary {
  background: #fff;
  color: #2563eb;
  font-weight: 600;
}

.header .btn-primary:hover {
  background: #eff6ff;
}

.btn-secondary {
  background: #e5e7eb;
  color: #374151;
}

.btn-secondary:hover:not(:disabled) {
  background: #d1d5db;
}

.btn-edit {
  flex: 1;
  background: #2563eb;
  color: #fff;
}

.btn-edit:hover {
  background: #1d4ed8;
}

.btn-delete {
  flex: 1;
  background: #fee2e2;
  color: #dc2626;
}

.btn-delete:hover {
  background: #fecaca;
}

.btn-danger {
  background: #dc2626;
  color: #fff;
}

.btn-danger:hover {
  background: #b91c1c;
}

/* Кнопка в модальному вікні */
.modal .btn-primary {
  background: #2563eb;
  color: #fff;
  font-weight: 600;
}

.modal .btn-primary:hover {
  background: #1d4ed8;
}

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
						42
Зм.	Арк.	№ докум.	Підпис	Дата		

```

}

/* Парінація */
.pagination {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 16px;
  margin: 32px 0;
}

.page-info {
  font-size: 14px;
  color: #6b7280;
}

/* Модальне вікно */
.modal-overlay {
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 16px;
  z-index: 100;
}

.modal {
  background: #fff;
  border-radius: 12px;
  padding: 24px;
  width: 100%;
  max-width: 560px;
  max-height: 90vh;
  overflow-y: auto;
}

.modal-small {
  max-width: 400px;
}

.modal-title {
  font-size: 20px;
  margin-bottom: 18px;
}

.confirm-text {
  color: #4b5563;
  margin-bottom: 20px;
}

/* Форма */
.form-group {
  margin-bottom: 14px;
  display: flex;
  flex-direction: column;
}

.form-row {
  display: flex;
  gap: 12px;
}

.form-row .form-group {
  flex: 1;
}

.form-group label {
  font-size: 13px;
  font-weight: 600;
}

```

					2026.KP.122.321.10.00.00 ПЗ	Арк.
						43
Зм.	Арк.	№ докум.	Підпис	Дата		

```

    color: #374151;
    margin-bottom: 5px;
}

.form-group input,
.form-group textarea,
.form-group select {
    padding: 9px 12px;
    border: 1px solid #d1d5db;
    border-radius: 8px;
    font-size: 14px;
    font-family: inherit;
}

.form-group input:focus,
.form-group textarea:focus,
.form-group select:focus {
    outline: none;
    border-color: #2563eb;
}

.error {
    color: #dc2626;
    font-size: 12px;
    margin-top: 4px;
}

.modal-actions {
    display: flex;
    gap: 10px;
    margin-top: 8px;
}

```

					2026.КР.122.321.10.00.00 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		44